What Java Is

Java is a general-purpose, object-oriented, class-based programming language created by Sun Microsystems (James Gosling, released 1995), now owned by Oracle. Its core philosophy is **"Write Once, Run Anywhere" (WORA)** — Java code compiles to bytecode that runs on the **JVM (Java Virtual Machine)**, making it platform-independent.

Core Architecture

- **JDK (Java Development Kit)** — tools to write and compile Java (includes compiler `javac`, debugger, etc.)
- **JRE (Java Runtime Environment)** — libraries + JVM needed to *run* Java programs
- **JVM (Java Virtual Machine)** — executes bytecode (`.class` files), handles memory management, garbage collection
- Compilation flow: `.java` → `javac` → `.class` (bytecode) → JVM interprets/JIT-compiles → machine code

Object-Oriented Pillars

1. **Encapsulation** — bundling data + methods, controlling access via `private`/`public`/`protected`
2. **Inheritance** — `extends` keyword, code reuse via parent-child classes
3. **Polymorphism** — method overloading (compile-time) and overriding (runtime, via `@Override`)
4. **Abstraction** — `abstract` classes and `interface`s hide implementation details

Key Language Features

- **Strongly typed** — variables have fixed types (`int`, `String`, `boolean`, etc.)
- **Automatic memory management** — Garbage Collector reclaims unused objects (no manual `free()` like C/C++)
- **Exception handling** — `try/catch/finally`, checked vs unchecked exceptions
- **Collections Framework** — `List`, `Set`, `Map`, `ArrayList`, `HashMap`, `TreeMap`, etc.
- **Generics** — type-safe collections (`List<String>` instead of raw `List`)
- **Multithreading** — built-in via `Thread`, `Runnable`, `ExecutorService`, `synchronized`
- **Streams & Lambdas** (Java 8+) — functional-style processing: `list.stream().filter(...).map(...).collect(...)`
- **Records** (Java 14+) — concise immutable data classes

Common Java Ecosystem (relevant to your backend work)

- **Spring / Spring Boot** — dependency injection, REST APIs, auto-configuration
- **Spring Security + JWT** — authentication/authorization (like in your StayEase project)
- **Spring Data JPA / Hibernate** — ORM, maps Java objects to DB tables
- **Maven / Gradle** — build tools, dependency management
- **JUnit + Mockito** — unit testing and mocking (like your Employee Management System)
- **Swagger/OpenAPI** — API documentation

Memory Model (common interview topic)

- **Heap** — objects live here, garbage collected
- **Stack** — method calls, local variables, per-thread
- **Method Area / Metaspace** — class metadata

Why It Matters for Interviews

Common Java interview themes: JVM internals, `String` immutability & the String Pool, `==` vs `.equals()`, `HashMap` internal working (buckets, hashing, collision handling), garbage collection algorithms, checked vs unchecked exceptions, and multithreading concepts (`synchronized`, `volatile`, deadlocks).

Want me to go deeper into any one area — like JVM internals, Collections Framework, or Spring Boot specifically — since those come up a lot in the backend interviews you've been prepping for?